

ASSESSMENT 1

CO3



NAME:

V.MAHALAKSHMI



REG NO:

192521181



COURSE CODE:

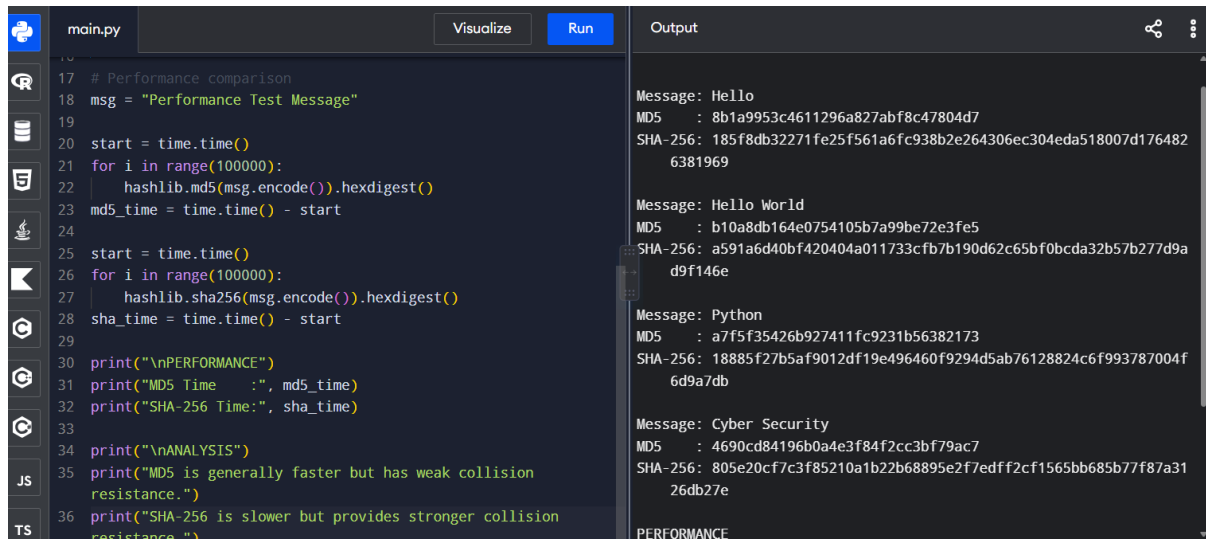
CSA5113



COURSE:

CRYPTOGRAPHY AND
NETWORK SECURITY

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.



```
main.py Visualize Run Output
17 # Performance comparison
18 msg = "Performance Test Message"
19
20 start = time.time()
21 for i in range(100000):
22     hashlib.md5(msg.encode()).hexdigest()
23 md5_time = time.time() - start
24
25 start = time.time()
26 for i in range(100000):
27     hashlib.sha256(msg.encode()).hexdigest()
28 sha_time = time.time() - start
29
30 print("\nPERFORMANCE")
31 print("MD5 Time :", md5_time)
32 print("SHA-256 Time:", sha_time)
33
34 print("\nANALYSIS")
35 print("MD5 is generally faster but has weak collision
36 print("SHA-256 is slower but provides stronger collision
37     resistance.")
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
Message: Hello
MD5 : 8b1a9953c4611296a827abf8c47804d7
SHA-256: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d176482
6381969

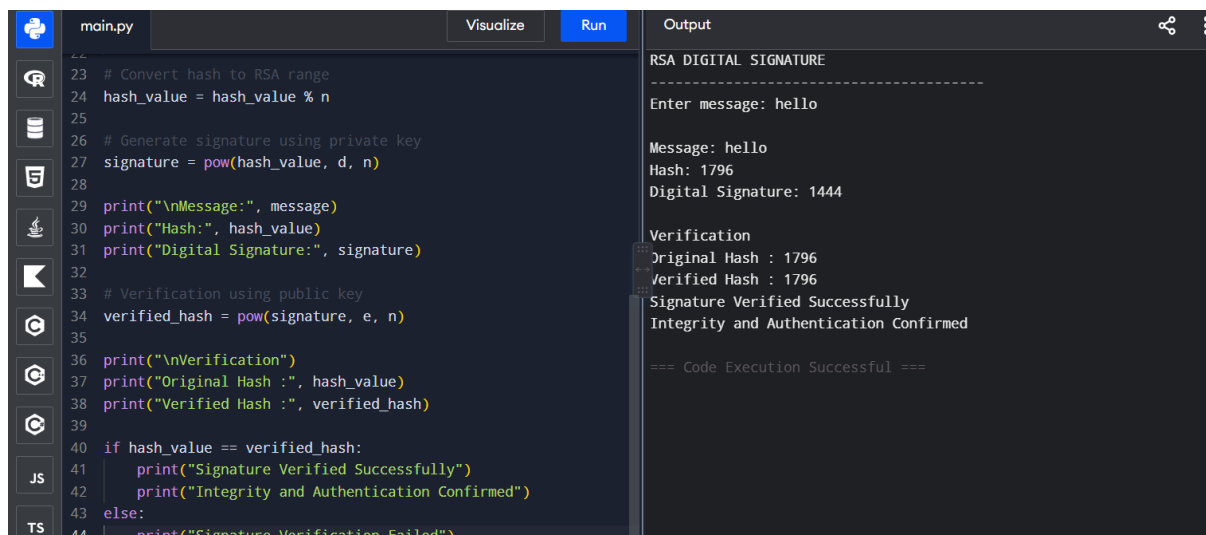
Message: Hello World
MD5 : b10a8db164e0754105b7a99be72e3fe5
SHA-256: a591a6d40bf420404a011733cfb7b190d62c65bf0bca32b57b277d9a
d9f146e

Message: Python
MD5 : a7f5f35426b927411fc9231b56382173
SHA-256: 18885f27b5af9012df19e496460f9294d5ab76128824c6f993787004f
6d9a7db

Message: Cyber Security
MD5 : 4690cd84196b0a4e3f84f2cc3bf79ac7
SHA-256: 805e20cf7c3f85210a1b22b68895e2f7edff2cf1565bb685b77f87a31
26db27e

PERFORMANCE
```

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.



```
main.py Visualize Run Output
23 # Convert hash to RSA range
24 hash_value = hash_value % n
25
26 # Generate signature using private key
27 signature = pow(hash_value, d, n)
28
29 print("\nMessage:", message)
30 print("Hash:", hash_value)
31 print("Digital Signature:", signature)
32
33 # Verification using public key
34 verified_hash = pow(signature, e, n)
35
36 print("\nVerification")
37 print("Original Hash :", hash_value)
38 print("Verified Hash :", verified_hash)
39
40 if hash_value == verified_hash:
41     print("Signature Verified Successfully")
42     print("Integrity and Authentication Confirmed")
43 else:
44     print("Signature Verification Failed")
```

```
RSA DIGITAL SIGNATURE
-----
Enter message: hello

Message: hello
Hash: 1796
Digital Signature: 1444

Verification
Original Hash : 1796
Verified Hash : 1796
Signature Verified Successfully
Integrity and Authentication Confirmed

=== Code Execution Successful ===
```

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

```

main.py Visualize Run Output
33 received_hmac = hmac.new(
34     message.encode(),
35     key.encode(),
36     hashlib.sha256
37 ).hexdigest()
38
39 print("\nSIMPLE HASH CHECK")
40
41 if received_hash == simple_hash:
42     print("Hash Verification Successful")
43 else:
44     print("Hash Verification Failed")
45
46 print("\nHMAC CHECK")
47
48 if hmac.compare_digest(received_hmac, hmac_value):
49     print("HMAC Verification Successful")
50     print("Message is Authentic and Unmodified")
51 else:
52     print("HMAC Verification Failed")
53
54 print("\nANALYSIS")
55 print("Simple hash provides integrity checking.")
56 print("HMAC provides integrity and authentication using a secret key.")

```

Output

```

HMAC AND SIMPLE HASH COMPARISON
-----
Enter message: hello
Enter secret key: secret123

Original Message: hello
SHA-256 Hash: 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e7304
3362938b9824
HMAC-SHA256: 6edb7e533ff92551f967bc02167127b93850a6ba3433efaca8bcf
d24783b160d

Enter received message: hello

SIMPLE HASH CHECK
Hash Verification Successful

HMAC CHECK
HMAC Verification Successful
Message is Authentic and Unmodified

ANALYSIS
Simple hash provides integrity checking.
HMAC provides integrity and authentication using a secret key.

```

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

```

main.py Visualize Run Output
20 else:
21     print("Ticket Generated: ", ticket)
22
23 # Send ticket to server
24 print("\nClient sends ticket to server...")
25
26 # Server validates ticket
27 current_time = int(time.time())
28
29 if current_time - timestamp <= 30:
30     print("Server: Ticket Valid")
31     print("Authentication Successful")
32     print("Access Granted")
33 else:
34     print("Server: Ticket Expired")
35     print("Access Denied")
36
37 print("\nANALYSIS")
38 print("The ticket contains a timestamp.")
39 print("The short ticket lifetime helps prevent replay attacks.")
40 print("Real Kerberos also uses encryption and session keys.")

```

Output

```

SIMPLIFIED KERBEROS AUTHENTICATION
-----
Enter username: alice
Enter password: 1234

Authentication Server: Login Successful
Ticket Generated:
ea81d1c978510037fb39bf61fe71417c5c799786734ac92bd2a4878fc7ff39f
9

Client sends ticket to server...
Server: Ticket Valid
Authentication Successful
Access Granted

ANALYSIS
The ticket contains a timestamp.
The short ticket lifetime helps prevent replay attacks.
Real Kerberos also uses encryption and session keys.

=== Code Execution Successful ===

```

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

```

main.py Visualize Run Output
37 else:
38     left = (
39         pow(g, s, p) *
40         pow(y, r, p)
41     ) % p
42
43 right = pow(g, h, p)
44
45 print("\nVerification")
46 print("Left Side :", left)
47 print("Right Side:", right)
48
49 if left == right:
50     print("Signature Verified Successfully")
51     print("Message Integrity and Authentication Confirmed")
52 else:
53     print("Signature Verification Failed")
54
55 print("\nANALYSIS")
56 print("ElGamal security is based on the discrete logarithm problem.")
57 print("It provides integrity and authentication.")
58 print("The private key must always remain secret.")

```

Output

```

ELGAMAL DIGITAL SIGNATURE
-----
Public Key:
p = 467
g = 2
y = 132

Enter message: hello

Message: hello
Hash: 224
Signature:
r = 8
s = 202

Verification
Left Side : 83
Right Side: 83
Signature Verified Successfully
Message Integrity and Authentication Confirmed

ANALYSIS
ElGamal security is based on the discrete logarithm problem.

```